Quadtrees on the GPU

Jonathan Dupuy, Jean-Claude Iehl, and Pierre Poulin

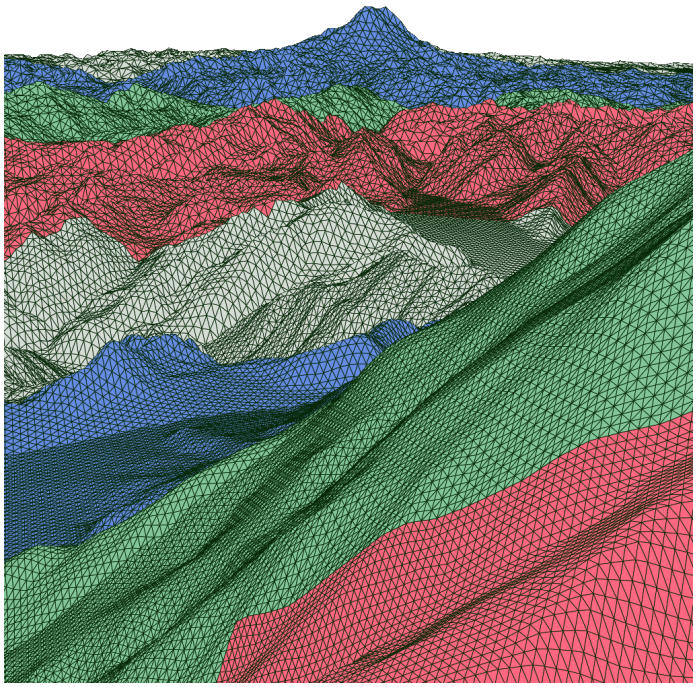

Figure 1.1. Crack-free, multiresolution terrain rendered from a quadtree implemented entirely on the GPU. The alternating colors show the different levels of the quadtree.

Abstract. We present a method to implement quadtrees on the GPU. It relies on linear trees, a pointer-free alternative to recursive trees, that represents nodes as bit codes. We explain how to update this data structure with node splitting and merging, and show how to render multiresolution, crack-free surfaces with frustum culling using hardware tessellation and a distance-based LOD selection criterion. Our implementation is both fast and lightweight, and runs asynchronously on the GPU, leaving CPU resources available for other tasks.

1.1 Introduction

Finding an appropriate mathematical representation for objects is a fundamental problem in computer graphics. Because they benefit from hardware acceleration, polygon meshes provide an appealing solution and are widely adopted by both interactive and high-quality renderers. But in order to maintain optimal rendering performance, special care must be taken to guarantee that each polygon projects into more than a few pixels. Below this limit, the Z-buffer starts aliasing, and the rasterizer's efficiency decreases drastically [AMD 10]. In the movie industry, this limitation is alleviated by using very high sampling rates, at the expense of dissuasive computation times. For applications that target interactivity however, such as video games or simulators, such overhead is unaffordable.

Another solution is to adapt the mesh to produce optimal on-screen polygons. If it can be simplified and/or enriched on the fly, it is said to be a scalable representation. Quadtrees provide such a scheme for grids, and have been used extensively since the early days of rendering, especially for terrain synthesis. Despite their wide adoption though, the first full GPU implementation was only introduced last year [Mistal 13]. This is due to two main reasons. First, quadtrees are usually implemented recursively, which makes them hard to handle in parallel by very nature (for instance, see the implementation of Strugar [Strugar 09]). Second, they require a T-junction removal system, which may result in an increase of draw calls [Andersson 07], adding non-negligible overhead to the CPU. As an alternative to quadtrees, instanced subdivision patches have been investigated [Cantlay 11, Fernandes and Oliveira 12]. Although such solutions can run entirely on the GPU, their scalability is currently too limited by the hardware.

In this article, we present a new implementation suitable for the GPU, which completely relieves the CPU without sacrificing scalability. In order to update the quadtree in parallel, we use the linear quadtree representation, which is presented in the next section. At render time, a square subdivision patch is instanced with a unique location and scale for each node. By employing a distance-based LOD criterion, we will show that T-junctions can be removed completely using a simple tessellation shader.

1.2 Linear Quadtrees

Linear quadtrees were introduced by Gargantini [Gargantini 82] as a non-recursive alternative to regular quadtrees. In this data structure, each node is represented by its subdivision level and a unique bit code, and only the leaves are stored in memory. The code is a concatenation of 2-bit words,

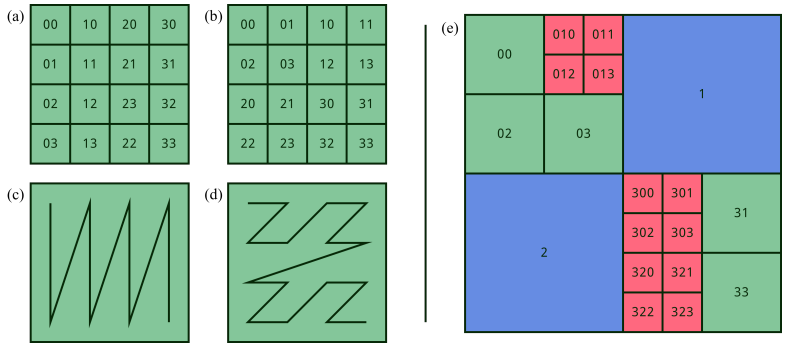

Figure 1.2. On the left, column major ordering (a) versus Morton ordering (b). The curve formed by Morton ordering (d) is often denoted as a Z-order curve. On the right, bit codes of a linear quadtree (e), built from the ordering of (b). The subdivision levels for the blue, green, and red nodes are respectively 1, 2, and 3.

each identifying the quadrant in which the node is located relatively to its parent. If these words are chosen so as to form a Z-order curve, then their concatenation forms a Morton code. In Figure 1.2 for instance, the codes 0, 1, 2, and 3 are mapped to the upper left (UL), upper right (UR), lower left (LL), and lower right (LR) quadrants, respectively. This last property allows direct translation to row/column major numbering (Figure 1.2, left) by bit de-interleaving the code.

```
// de-interleave a 32-bit word using the Shift-Or algorithm
uint lt_undilate_2(in uint x) {
        x = (x | (x >> 1u)) & 0x33333333;
        x = (x | (x >> 2u)) & 0x0F0F0F0F;
        x = (x | (x >> 4u)) & 0x00FF00FF;
        x = (x | (x >> 8u)) & 0x0000FFFF;
        return (x & 0x0000FFFF);
}

// retrieve column major position and level from a 32-bit word
void lt_decode_2_15(in uint key, out uint level, out uvec2 p) {
        level = key & 0xF;
        p.x = lt_undilate_2((key >> 4u) & 0x05555555);
        p.y = lt_undilate_2((key >> 5u) & 0x05555555);
}
```

Listing 1.1. GLSL node decoding routines. There are many ways to de-interleave a word. For a thorough overview of existing methods, we refer to the work of Raman and Wise [Raman and Wise 08].

Representation

Similarly to Shaffer [Shaffer 90], we use a single integer to represent each node. The rightmost bits are reserved for the subdivision level, and the adjacent bits are left for the bit code. Below is the bit representation of a 32-bit word, encoding the red node whose code is 321 in Figure 1.2. Bits irrelevant to the code are denoted by the '_' character.

```
msb                                     lsb
---- ---- ---- ---- ---- --11 1001 0011
```

The first 4 bits store the level ($0011_b = 3$ in this example), leaving the remaining 28 bits for the code. Using this configuration, a 32-bit word can store up to a $28/2 + 1 = 15$-level quadtree, which includes the root node. In Listing 1.1, we provide the procedures to retrieve the column major numbering from this representation. Naturally, more levels require longer words. Because longer integers are currently unavailable on many GPUs, we emulate them using integer vectors, where each component represents a 32-bit wide portion of the entire code. For more details, please see our implementation, where we provide a 30-level quadtree using the GLSL `uvec2` datatype.

```
// generate children nodes from a quadtree encoded
// in a 32-bit word
void lt_children_2_15(in uint key, out uint children[4]) {
        key = (++key & 0xF) | ((key & ~0xF) << 2u);
        children[0] = key;
        children[1] = key | 0x10;
        children[2] = key | 0x20;
        children[3] = key | 0x30;
}

// generate parent node from a quadtree encoded
// in a 32-bit word
uint lt_parent_2_15(in uint key) {
        return ((--key & 0xF) | ((key >> 2u) & 0x3FFFFFF0));
}
```

Listing 1.2. Splitting and merging procedures in GLSL.

Updating the Structure

When relying on quadtrees to generate progressive meshes, efficient updates are critical. Fortunately, node splitting and merging can be accomplished in a very straightforward manner with linear trees. The following bit representations match the parent of the node given in the previous example, and its four children.

```
          msb                                   lsb
parent: ____ ____ ____ ____ ____ ____ 1110 0010
node:   ____ ____ ____ ____ ____ __11 1001 0011
UL:     ____ ____ ____ ____ ____ 1110 0100 0100
UR:     ____ ____ ____ ____ ____ 1110 0101 0100
LL:     ____ ____ ____ ____ ____ 1110 0110 0100
LR:     ____ ____ ____ ____ ____ 1110 0111 0100
```

Note that compared to the node representation, the levels differ by one and the bit codes are either 2-bit expansions or contractions. The GLSL code to generate these representations is shown in Listing 1.2. It simply consists of an arithmetic addition, a bitshift, and logical operations, and is thus very cheap.

GPU Implementation

Managing a linear quadtree on the GPU requires two buffers and a geometry shader. During initialisation, we store the base hierarchy (we start with the root node only) in one of the buffers. Whenever the nodes must be updated, the buffer is iterated over by the geometry shader, set to write into the second buffer. If a node needs to be split, it emits four new words, and the original code is deleted. Conversely, when four nodes must merge, they are replaced by their parent's code. In order to avoid generating four copies of the same node in memory, we only emit the code once from the UL child, identified using the test provided in Listing 1.3. For the nodes that do not require any intervention, their code is simply emitted, unchanged.

It is clear that this approach maps very well to the GPU. Note however that an iteration only permits a single split or merge operation per node. Thus when more are needed, multiple buffer iterations should be performed. At each new iteration, we swap the first and second buffer so that the newest hierarchy is processed by the geometry shader. This strategy is also known as double, or ping-pong buffering. In our implementation, a single buffer iteration is performed at the beginning of each frame.

```
// check if node is the upper left child
// in a 32-bit word
bool lt_is_upper_left_2_15(in uint key) {
        return ((key & 0x30) == 0x00);
}
```

Listing 1.3. Determining if the node represents the UL child of its parent representation. Only the first 2-bit word of the Morton code has to be tested.

1.3 Scalable Grids on the GPU

So far, we have only discussed a parallel implementation of a quadtree. In this section, we present a complete OpenGL pipeline to extract a scalable grid from this representation, which in turn can be used to render terrains, as well as parametric surfaces.

LOD Function

Similarly to Strugar [Strugar 09], we essentially exploit the quadtree to build the final grid by drawing multiple copies of a static grid mesh. Each copy is associated with a node, so that it can be translated and scaled to the correct location. Listing 1.4 shows how the transformations are extracted from the codes and applied to each vertex of the instanced grid in a vertex shader.

In order to guarantee that the transformed vertices produce rasterizer-friendly polygons, a distance-based criterion can be used. Indeed, under perspective projection, the image plane size s at distance z from the camera can be determined analytically with the relation

$$s(z) = 2z \tan\left(\frac{\alpha}{2}\right)$$

where $\alpha \in (0, \pi]$ is the horizontal field of view. Thus if polygons scale proportionally to s, they are guaranteed to be of approximately constant size in screen space. Based on this observation, we derived the following routines to determine whether a node should be split or merged:

```
float z1 = distance(eye, node_center);
float z2 = distance(eye, parent_center);

if( k * node_size > s(z1) ) split();
else if( k * parent_size < s(z2) ) merge();
else keep();
```

Here, `k` is an arbitrary positive constant that controls the average number of on-screen cells. We found that using $\texttt{k} = 8$ produces good results, resulting in roughly 36 (6×6) cells in the image plane. Therefore, when the instanced grid has $n \times n$ polygons, the average on-screen polygon density is $36n^2$.

T-Junction Removal

The core of our meshing scheme closely resembles that of Strugar's [Strugar 09]; we both rely on mesh instancing and a distance-based LOD function to build a scalable grid. As such, his vertex morphing scheme can be used to avoid T-junctions. We suggest next an alternative solution, which relies on tessellation shaders.

```
layout(location = 0) in vec2 i_mesh; // instanced vertices
layout(location = 1) in uint i_node; // per instance data

// retrieve normalized coordinates and size of the cell
void lt_cell_2_15(in uint key, out vec2 p, out float size) {
        uvec2 pos;
        uint level;

        lt_decode_2_15(key, level, pos);
        size = 1.0 / float(1u << level); // in [0,1]
        p = pos * size; // in [0,1)
}

void main() {
        vec2 translation;
        float scale;

        // pass on vertex position and scale
        lt_cell_2_15(i_node, translation, scale);
        vec2 p = i_mesh * scale + translation;
        gl_Position = vec4(p, scale, 1.0);
}
```

Listing 1.4. GLSL vertex shader for rendering a scalable grid.

Before transforming the vertices of the instanced grid, we invoke a tessellation control shader and evaluate the LOD function at each edge of its polygons. We then divide their length by the computed value, and multiply the result by $\sqrt{2}$. This operation safely refines the grid polygons nearing neighbor cells of smaller resolution; its output is illustrated in Figure 1.3. Our GLSL tessellation control shader code is shown in Listing 1.5.

OpenGL Pipeline

Our OpenGL implementation consists of three passes. The first pass updates the quadtree, using the algorithm described in Section 1.2. After the tree has been updated, a culling kernel iterates over the nodes, testing their intersection with the view frustum. Note that we can reuse the buffer holding the nodes before the first pass to store the results. Since we only have access to the leaf nodes, no hierarchical optimizations can be performed, so each node is tested independently. This scheme still performs very well nonetheless, as it is very adequate for the parallel architecture of the GPU. Finally, the third pass renders the scalable grid to the back buffer, using the algorithms described earlier in this section. The full pipeline is diagrammed in Figure 1.4.

```
layout(vertices = 4) out; // quad patches
uniform vec3 u_eye_pos; // eye position
void main() {
        // get data from vertex stage
        vec4 e1 = gl_in[gl_InvocationID].gl_Position;
        vec4 e2 = gl_in[(gl_InvocationID+1)%4].gl_Position;

        // compute edge center and LOD function
        vec3 ec = vec3(0.5 * e1.xy + 0.5 * e2.xy, 0);
        float s = 2.0 * distance(u_eye_pos, ec) * u_tan_fov;

        // compute tessellation factors (1 or 2)
        const float k = 8.0;
        float factor = e1.z * k * sqrt(2.0) / s;
        gl_TessLevelOuter[gl_InvocationID] = factor;
        gl_TessLevelInner[gl_InvocationID%2] = factor;

        // send data to subsequent stages
        gl_out[gl_InvocationID].gl_Position =
                gl_in[gl_InvocationID].gl_Position;
}
```

Listing 1.5. GLSL tessellation control shader for rendering a scalable grid.

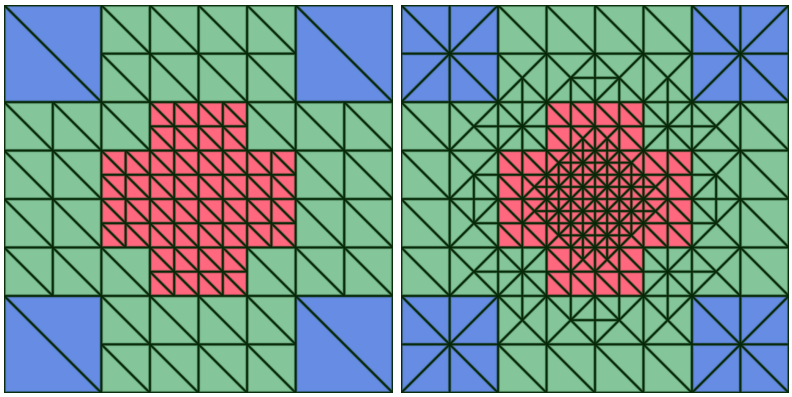

Figure 1.3. Multiresolution mesh (left) before and (right) after tessellation. T-junctions have been completely removed.

Results

To demonstrate the effectiveness of our method, we wrote a renderer for terrains, and another one for parametric surfaces. Some results can be seen in Figures 1.1 and 1.5. In Table 1.1, we give the CPU and GPU timings of a zoom-in/zoom-out sequence using the terrain renderer at 720p.

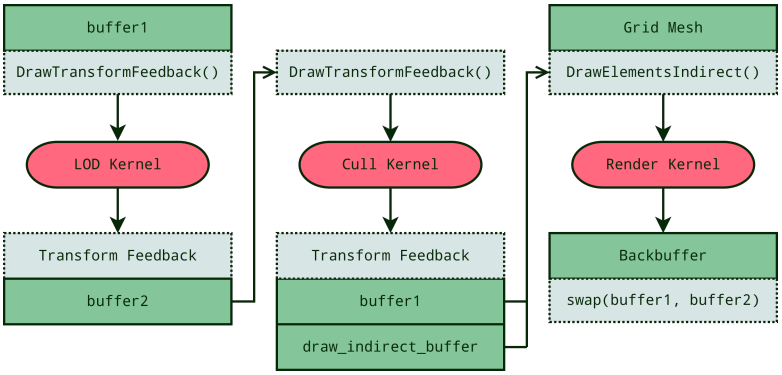

Figure 1.4. OpenGL pipeline of our method.

The camera's orientation was fixed, looking downwards, so that the terrain would occupy the whole framebuffer, thus maintaining constant rasterization activity. The testing platform is an Intel Core i7-2600K, running at 3.40 GHz, and a Radeon 7950 GPU. Note that the CPU activity only consists of OpenGL uniform variables and driver management. On current implementations, such tasks run asynchronously to the GPU.

Kernel	CPU (ms)	GPU (ms)	CPU stdev	GPU stdev
LOD	0.096	0.067	0.039	0.026
Cull	0.068	0.077	0.035	0.032
Render	0.064	1.271	0.063	0.080

Table 1.1. CPU and GPU timings and their respective standard deviation over a zoom-in sequence of 1000 frames.

As demonstrated by the reported numbers, the performance of our implementation is both fast and stable. Naturally, the average GPU rendering time depends on how the terrain is shaded. In our experiment, we use solid wireframe shading [Gateau 07] which, despite requiring a geometry shader, is fairly cheap.

1.4 Discussion

This section provides a series of answered questions, which should give better insights on the properties and possible extensions of our implementation.

How much memory should be allocated for the buffers containing the nodes?

This depends on the α and `k` values, which control how fast distant nodes should be merged. The buffers should be able to store at least $3 \times \text{max\_level} + 1$ nodes, and do not need to exceed a capacity of $4^{\text{max\_level}}$ nodes. The lower bound corresponds to a perfectly restricted quadtree, where each neighboring cell differs by one level of subdivision at most. The higher bound gives the number of cells at the finest subdivision level.

Is the quadtree prone to floating-point precision issues?

There are no issues regarding the tree structure itself, as each node is represented with bit sequences only. However, problems may occur when extracting the location and scale factors applied to the instanced grid during the rendering pass, in Listing 1.4. The 15-level quadtree does not have this issue, but higher levels will, eventually. A simple solution to delay the problem on OpenGL4+ hardware is to use double precision, which should provide sufficient comfort for most applications.

Could fractional tessellation be used to produce continuous subdivisions?

If the T-junctions are removed with the CDLOD morphing function [Strugar 09], any mode can be used. Our proposed alternative approach does not have restrictions regarding the inner tessellation levels, but the outer levels must be set to exact powers of two. Therefore, fractional odd tessellation will not work. The even mode can be used though.

Are there any significant reasons to use tessellation rather than the CDLOD morphing function for T-junctions?

The only advantage of tessellation is flexibility, since arbitrary factors can be used. For some parametric surfaces, tessellation turned out to be essential because the quadtree we computed was not necessarily restricted. The phenomenon is visible in Figure 1.5, on the blue portions of the trefoil knot. If we had used the CDLOD method, a crack would have occurred. For terrains, the distance-based criterion ensures that the nodes are restricted, so both solutions are valid.

There are two ways to increase polygon density. Either use the GPU tessellator, or refine the instanced grid. Which approach is best?

This will naturally depend on the platform. Our renderers provide tools to modify both the grid and the GPU tessellation values, so that their impact can be thoroughly measured. On the Radeon 7950, we observed that tessellation levels could be increased up to a factor of 8 without performance penalties, as long as the average on-screen polygon size remained reasonable.

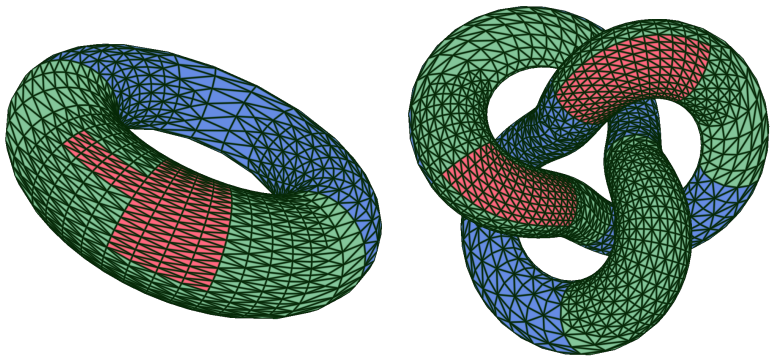

Figure 1.5. Crack-free, multiresolution parametric surfaces produced on the GPU with our renderer.

1.5 Conclusion

We have presented a novel implementation for quadtrees running completely on the GPU. It takes advantage of linear trees to alleviate the recursive nature of common tree implementations, offering a simple and efficient data structure for parallel processors. While this work focuses on quadtrees, this representation can also be used for higher dimensional trees, such as octrees. Using the same distance-based criterion, the *Transvoxel* algorithm [Lengyel 10] could be employed to produce a crack-free volume extractor, running entirely on the GPU. We expect such an implementation to be extremely fast as well.

Acknowledgements

This work was partly funded in Canada by GRAND and in France by ANR. Jonathan Dupuy acknowledges additional financial support from an Explo'ra Doc grant from "région Rhône-Alpes", France.

Bibliography

[AMD 10] AMD. "Tessellation for all.", 2010. Available online (http://community.amd.com/community/amd-blogs/game/blog/2010/11/29/tessellation-for-all).

[Andersson 07] Johan Andersson. "Terrain rendering in frostbite using procedural shader splatting." In *ACM SIGGRAPH 2007 courses*, *SIGGRAPH '07*, pp. 38–58. ACM, 2007. Available online (http://doi.acm.org/10.1145/1281500.1281668).

[Cantlay 11] Iain Cantlay. "DirectX 11 terrain tessellation.", 2011. Available online (http://developer.nvidia.com/sites/default/files/akamai/gamedev/files/sdk/11/TerrainTessellation_WhitePaper.pdf).

[Fernandes and Oliveira 12] António Ramires Fernandes and Bruno Oliveira. "GPU tessellation: we still have a LOD of terrain to cover." In *OpenGL Insights*, edited by Patrick Cozzi and Christophe Riccio, pp. 145–162. CRC Press, 2012.

[Gargantini 82] Irene Gargantini. "An effective way to represent quadtrees." *Communications of the ACM* 25:12 (1982), 905–910.

[Gateau 07] Samuel Gateau. "Solid wireframe.", 2007. Available online (http://developer.download.nvidia.com/SDK/10.5/direct3d/Source/SolidWireframe/Doc/SolidWireframe.pdf).

[Lengyel 10] Eric Lengyel. "Voxel-based terrain for real-time virtual simulations." Ph.D. thesis, University of California at Davis, 2010.

[Mistal 13] Benjamin Mistal. "GPU terrain subdivision and tessellation." In *GPU Pro 4: Advanced Rendering Techniques*, edited by Wolfgang Engel. AK Peters / CRC Press, 2013. Available online (http://www.crcpress.com/product/isbn/9781466567436).

[Raman and Wise 08] Rajeev Raman and David Stephen Wise. "Converting to and from dilated integers." *IEEE Transactions on Computers* 57:4 (2008), 567–573.

[Shaffer 90] Clifford A. Shaffer. "Bit interleaving for quad or octrees." In *Graphics Gems*, edited by Andrew S. Glassner, pp. 443–447. Academic Press, 1990. Available online (http://dl.acm.org/citation.cfm?id=90767.90895).

[Strugar 09] Filip Strugar. "Continuous distance-dependent level of detail for rendering heightmaps." *Journal of Graphics, GPU, and Game Tools* 14:4 (2009), 57–74.